

算术运算类指令

算术运算

对数据进行加减乘除

基本的数据处理方法

加减运算有“和”或“差”的结果外，还有进借位、溢出等状态标志，也是结果的一部分

注意算术运算类指令对标志的影响

掌握：加法和减法指令

熟悉：乘法和除法指令

理解：零位扩展和符号扩展

3.2 算术运算类指令

3.2.1 状态位

- ① 进位位 (CF) : **无符号数**加减是否超过表示范围, 有进位和借位 $CF=1$; 反之, $CF=0$, 结果仍正确。
- ② 溢出位 (OF) : **有符号数**加减是否超过范围, 符号相同的两数运算 (两个正数加得负数, 两个负数相加得到正数), 得到结果符号与操作数不同, 溢出。结果错误。
- ③ 奇偶位 (PF) : 低8位中1的个数, 偶数个, $PF=1$ 。
- ④ 零标志位 (ZF) : 结果为0, $ZF=1$ 。
- ⑤ 符号位 (SF) : 最高位为符号位, 即SF。



3. 2. 2 算术运算指令

加法指令：ADD，INC，ADC

减法指令：SUB，DEC，SBB，NEG

汇编语言程序设计

进位和溢出标志

状态标志

- 状态标志是处理器最基本的标志
 - ▶ 一方面：作为加减运算和逻辑运算的辅助结果
 - ▶ 另一方面：构成各种条件，实现程序分支

15	12	11	10	9	8	7	6	5	4	3	2	1	0
		OF	DF	IF	TF	SF	ZF	0	AF	0	PF	1	CF

8086的标志

进位标志CF (Carry Flag)

- 当加减运算结果的最高有效位有进位（加法）或借位（减法）时，进位标志置1，即**CF=1**；否则**CF=0**
- 针对无符号整数，判断加减结果是否超出表达范围

无符号整数的表达范围

N位	8位	16位	32位
$0 \sim 2^N - 1$	$0 \sim 255$	$0 \sim 65535$	$0 \sim 2^{32} - 1$

进位标志CF举例—1

8位二进制

$$\begin{array}{r} 00111010 \\ + 01111100 \\ \hline 10110110 \end{array}$$

十六进制

$$\begin{array}{r} 3A \\ + 7C \\ \hline B6 \end{array}$$

十进制

$$\begin{array}{r} 58 \\ + 124 \\ \hline 182 \end{array}$$

$0 < 182 < 255$



结果: 182, 无进位CF=0

进位标志CF举例—2

8位二进制

$$\begin{array}{r} 10101010 \\ + 01111100 \\ \hline 100100110 \end{array}$$

十六进制

$$\begin{array}{r} AA \\ + 7C \\ \hline 126 \end{array}$$

十进制

$$\begin{array}{r} 170 \\ + 124 \\ \hline 294 = 256 + 38 \end{array}$$

255 < 294

结果38，有进位 CF = 1

低位部分

溢出标志OF (Overflow Flag)

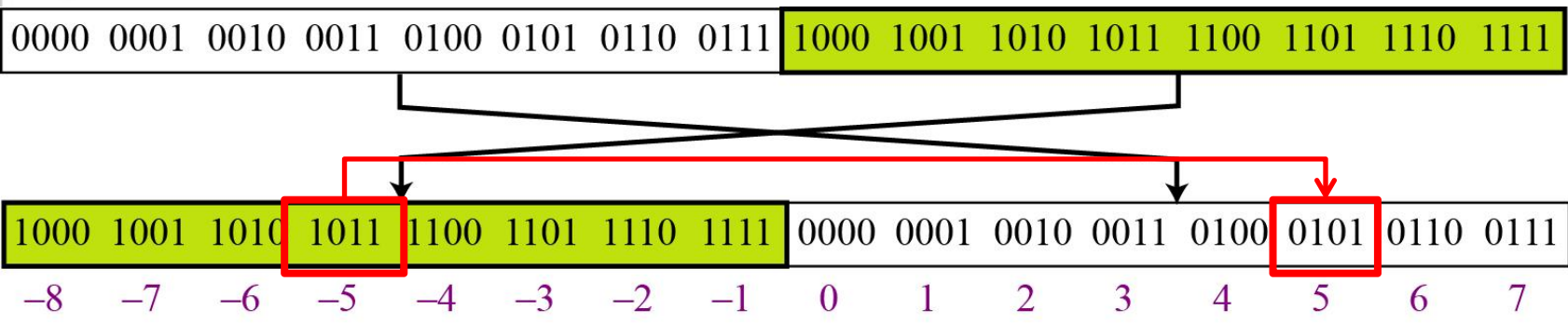
- 有符号数加减结果有溢出，则 **OF=1**；否则 **OF=0**
- 针对有符号整数，判断加减结果是否超出表达范围
有符号整数（补码）的表达范围

N位	8位	16位	32位
$-2^{N-1} \sim 2^{N-1}-1$	$-128 \sim 127$	$-32768 \sim 32767$	$-2^{31} \sim 2^{31}-1$

杯中水已满，再加就溢出！

二进制补码表示法

- 现代计算机使用**二进制补码**（two's complement）表示法储存**有符号整数**。
- 最左边位表示整数的符号。如果该位为 0，则此整数为正；如果该位为 1，则此整数为负。



溢出标志OF举例—1

8位二进制

$$\begin{array}{r} 00111010 \\ + 01111100 \\ \hline 10110110 \end{array}$$

十六进制

$$\begin{array}{r} 3A \\ + 7C \\ \hline B6 \end{array}$$

十进制

$$\begin{array}{r} 58 \\ + 124 \\ \hline 182 \end{array}$$

127 < 182

结果: -74, 超范围 OF=1

错误

溢出标志OF举例—2

8位二进制	十六进制	十进制
$\begin{array}{r} 10101010 \\ + 01111100 \\ \hline 100100110 \end{array}$	$\begin{array}{r} AA \\ + 7C \\ \hline 126 \end{array}$	$\begin{array}{r} -86 \\ + 124 \\ \hline 38 \end{array}$

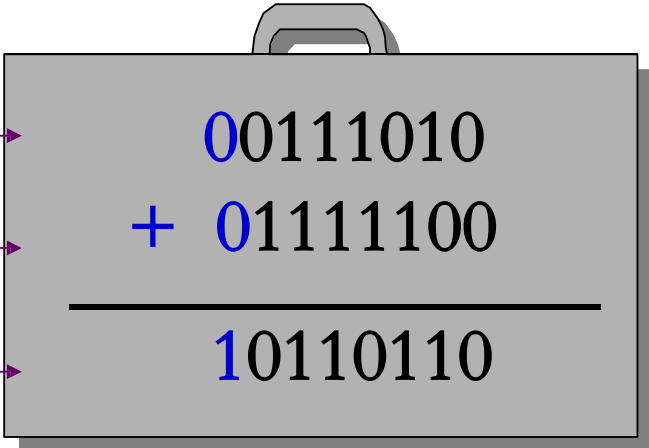
-128 < **126** < 127

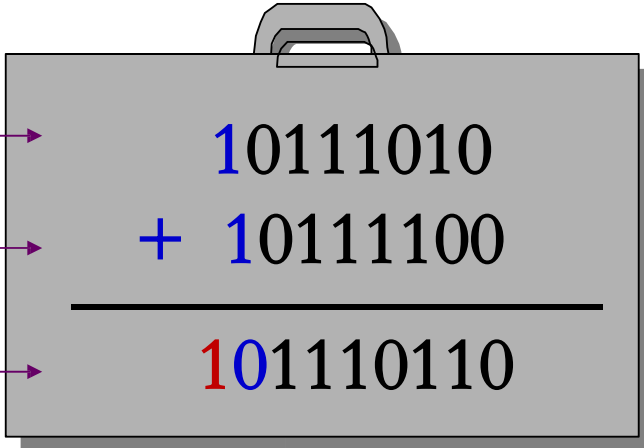


结果: **38**, 范围内 **OF=0**

溢出标志的人工判断

- 只有当两个相同符号数相加(含两个不同符号数相减)
- 而运算结果的符号与原数据符号相反时, 产生溢出


$$\begin{array}{r} \text{正数} \rightarrow 00111010 \\ + \text{正数} \rightarrow 01111100 \\ \hline \text{负数} \rightarrow 10110110 \end{array}$$


$$\begin{array}{r} \text{负数} \rightarrow 10111010 \\ + \text{负数} \rightarrow 10111100 \\ \hline \text{正数} \rightarrow 101110110 \end{array}$$

- 其他情况下, 不会产生溢出

进位和溢出的区别

- 进位标志反映无符号整数运算结果是否超出范围
 - ▶ 有进位，加上进位或借位后运算结果仍然正确
- 溢出标志反映有符号整数运算结果是否超出范围
 - ▶ 有溢出，运算结果已经不正确
- 处理器按照无符号整数求得结果
 - ▶ 设置进位标志CF
 - ▶ 设置溢出标志OF

程序员决定
无符号数，关心进位
有符号数，注意溢出

CF (Carry Flag)

进位标志 / 借位标志

- 对加法：看最高位是否产生进位
- 对减法：看最高位是否产生借位
- 主要用于判断：无符号数运算

OF (Overflow Flag)

溢出标志

- 看结果是否超出有符号数表示范围
- 主要用于判断：有符号数（补码）运算
- 本质上关注“符号是否异常翻转”

一句话：CF 管无符号，OF 管有符号。

CF 和 OF 的区别对照表

Comparison

标志位	中文含义	关注角度	置 1 的条件	适合判断	课堂记忆点
CF	进位/借位标志	无符号数	加法最高位有进位 减法最高位有借位	无符号运算是 否越界	看“位”
OF	溢出标志	有符号数（补码）	结果超出有符号数表示 范围	有符号运算是 否溢出	看“符号”

CF 看进位借位，OF 看符号越界。
或者更短：CF 管无符号，OF 管有符号。

例 1: CF=1, OF=0

```
1 mov al, 0FFh
2 add al, 01h
```

为什么?

$\text{FFh} + 01\text{h} = 100\text{h}$

8 位结果保留低 8 位, 得到 00h。

- 最高位有进位 $\rightarrow \text{CF} = 1$
- 按有符号数看: $-1 + 1 = 0 \rightarrow \text{OF} = 0$

例 2: CF=0, OF=1

```
1 mov al, 7Fh
2 add al, 01h
```

为什么?

$7\text{Fh} = 127$ 。

$127 + 1 = 128$, 8位补码结果变成80h (-128)。

- 没有向更高位进位 $\rightarrow \text{CF} = 0$
- 正数 + 正数 得到负数 $\rightarrow \text{OF} = 1$

128超过-128~127的范围, OF=1

结论: CF 和 OF 可能不同步, 不能混为一谈。

为什么 CF=1, OF=0

Worked Example

题目

```
1 mov eax, AAFF3322H
2 sub eax, 0BB000000H
```

为什么 CF = 1

从无符号数看：

AAFF3322H < BB000000H

所以做减法时发生最高位的借位，CF = 1。

运算结果

AAFF3322H - BB000000H = EFFF3322H

为什么 OF = 0

从有符号数看：

结果 EFFF3322H 仍在 32 位补码可表示范围内，
没有发生有符号溢出，所以 OF = 0。

两个负数相减（范围都在 $-2^{N-1} \sim -1$ ），无论如何都不可能超过 $-2^{N-1} \sim 2^{N-1}-1$ ，例如N=8， $-1 - (-128) = 127$ ； $(-128) - (-1) = -127$

同一条减法指令中：CF 反映无符号借位，OF 反映有符号溢出。



福州大学
FUZHOU UNIVERSITY

汇编语言程序设计

零标志、符号标志和奇偶标志

状态标志

- 状态标志是处理器最基本的标志
 - ▶ 一方面：作为加减运算和逻辑运算的辅助结果
 - ▶ 另一方面：构成各种条件，实现程序分支

15	12	11	10	9	8	7	6	5	4	3	2	1	0
		OF	DF	IF	TF	SF	ZF	0	AF	0	PF	1	CF

8086的标志

零标志ZF (Zero Flag)

➤ 运算结果为0，则ZF=1，否则ZF=0

结果是0，ZF标志不是0！

举例

➤ 8位二进制数相加：结果不是0，ZF=0

$$00111010 + 01111100 = 10110110$$

➤ 8位二进制数相加：结果是0，ZF=1

$$10000100 + 01111100 = [1]00000000$$

进位

结果

符号标志SF (Sign Flag)

➤ 运算结果最高位为1，则**SF=1**；否则**SF=0**

最高位=符号位=SF

举例

➤ 8位二进制数相加：最高位=1，**SF=1**

$$00111010 + 01111100 = 10110110$$

➤ 8位二进制数相加：最高位=0，**SF=0**

$$10000100 + 01111100 = [1]00000000$$

进位 结果

奇偶标志PF (Parity Flag)

- 当运算结果最低字节中“1”的个数为零或偶数时，
PF=1；否则**PF=0**

仅最低8位“1”的个数

举例

- 8位二进制数相加：“1”的个数为5个，**PF=0**

$$00111010 + 01111100 = 10110110$$

- 8位二进制数相加：“1”的个数为0个，**PF=1**

$$10000100 + 01111100 = [1]00000000$$

进位

结果

加减运算结果同时影响状态标志

8位加法运算及结果	CF	OF	ZF	SF	PF
$00111010 + 01111100 = [0]10110110$	0	1	0	1	0
$10101010 + 01111100 = [1]00100110$	1	0	0	0	0
$10000100 + 01111100 = [1]00000000$	1	0	1	0	1

- ① 当加减运算结果的最高有效位有进位（加法）或借位（减法）时，进位标志置1，即CF=1；否则CF=0
- ② 有符号数加减结果有溢出，则OF=1；否则OF=0
- ③ 运算结果为0，则ZF=1，否则ZF=0
- ④ 运算结果最高位为1，则SF=1；否则SF=0
- ⑤ 当运算结果最低字节中“1”的个数为零或偶数时，PF=1；否则PF=0

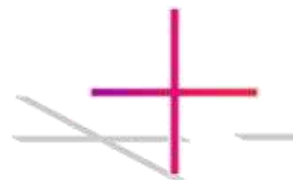


- 需要关注对标志影响的主要指令： 加减运算指令、逻辑运算指令、移位指令等
- 只用于影响标志的特殊指令1： 比较指令CMP
 - 进行减法运算
 - 用于判断两个数据大小、是否相等
- 只用于影响标志的特殊指令2： 测试指令TEST
 - 进行逻辑与运算
 - 用于判断某位为0或为1等

算术运算指令-加法指令

加法指令

- 加法指令 **ADD**
- 带进位加法指令 **ADC**
- 增量指令 **INC**



- ▶ 除**INC**不影响进位标志**CF**外
 - ▶ 其他指令按定义影响全部状态标志位
- 即，按照运算结果相应设置各个状态标志为0或为1



加法指令ADD

- 目的操作数加上源操作数，和送到目的操作数

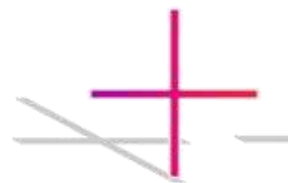
ADD reg, imm/reg/mem

;reg ← reg + imm/reg/mem

ADD mem, imm/reg

;mem ← mem + imm/reg

- ▶ 按照定义影响状态标志位
- ▶ 支持8位(字节)、16位(字)和32位(双字)数据运算





- ① 当加减运算结果的最高有效位有进位（加法）或借位（减法）时，进位标志置1，即CF=1；否则CF=0
- ② 有符号数加减结果有溢出，则OF=1；否则OF=0
- ③ 运算结果为0，则ZF=1，否则ZF=0
- ④ 运算结果最高位为1，则SF=1；否则SF=0
- ⑤ 当运算结果最低字节中“1”的个数为零或偶数时，PF=1；否则PF=0

mov eax,0aaff7348h

add al,27h

;8位加法

;最高位是D₇

加法之前 EAX	AAFF7348H
8位加法	+27H
加法之后 EAX	AAFF736FH

0110 1111

OF=0, SF=0, ZF=0, PF=1, CF=0

- ① 当加减运算结果的最高有效位有进位（加法）或借位（减法）时，进位标志置1，即CF=1；否则CF=0
- ② 有符号数加减结果有溢出，则OF=1；否则OF=0
- ③ 运算结果为0，则ZF=1，否则ZF=0
- ④ 运算结果最高位为1，则SF=1；否则SF=0
- ⑤ 当运算结果最低字节中“1”的个数为零或偶数时，PF=1；否则PF=0

add ax, 3ffffh

;16位加法

;最高位是D₁₅

加法之前EAX	AAFF736FH
16位加法 加	+3FFFFH
法之后EAX	AAFFB36EH

1011 0011 0110 1110

OF=1, SF=1, ZF=0, PF=0, CF=0

ADD 指令：32位加法

add eax,88000000h

;32位加法

;最高位是D₃₁

加法之前 EAX	AAFFB36EH
32位加法	+88000000H
加法之后 EAX	32FFB36EH

0011 0010 1111 1111 1011 0011 0110 1110

OF=1, SF=0, ZF=0, PF=0, CF=1

大写字母转换为小写可以用ADD

;确认是大写字母, 加20H为小写字母

add al,20h ;20H='a' - 'A'

;20H=' '(空格字符)

小写=大写+20H

大写=小写-20H

字母

ASCII

值 41H

'A'

42H

'B'

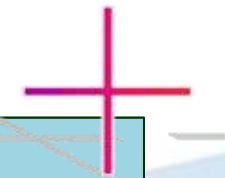
'a'

61H

'b'

62H

- **ADD** 指令是最基本的加法指令
- 支持8、16和32位加法运算
 - ▶ 并按照8、16和32位相应影响状态标志
 - ▶ 但**PF**标志只利用低8位结果



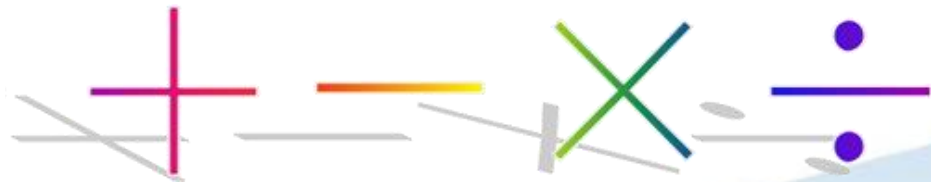
```
ADD dest, src  
; dest ← dest + src
```

汇编语言程序设计

SUB指令

算术运算类指令

- 算术运算对数据进行加减乘除
- 这是基本的数据处理方法
- 注意，加减运算有“和”或“差”的结果外，
还有进借位、溢出等状态标志，也是结果的一部分



减法指令

- 减法指令 **SUB**
- 带借位减法指令 **SBB**
- 减量指令 **DEC**
- 求补指令 **NEG**
- 比较指令 **CMP**
 - 除**DEC**不影响**CF**标志外
 - 其他按定义影响全部状态标志位



减法指令SUB (subtract)

- 目的操作数减去源操作数，差送到目的操作数

SUB reg,imm/reg/mem

;reg ← reg — imm/reg/mem

SUB mem,imm/reg

;mem ← mem — imm/reg

- ▶ 按照定义影响状态标志位
- ▶ 支持8位(字节)、16位(字)和32位(双字)数据运算

SUB 指令：8 位减法

```
mov eax,0aaff7348h
```

```
sub al,27h
```

;8位减法

;最高位是D₇

减法之前EAX	AAFF7348H
8位减法	- 27H
减法之后EAX	AAFF7321H

0010 0001

OF=0, SF=0, ZF=0, PF=1, CF=0

SUB 指令：16 位减法

sub ax,3fffh

;16 位减法

;最高位是

D_{15}

减法之前 **EAX**

AAFF7321H

16 位减法 减

- 3FFFH

法之后 **EAX**

AAFF3322H

0011 0011 0010 0010

OF=0, SF=0, ZF=0, PF=1, CF=0

SUB 指令：32位减法

sub eax,0bb000000h

;32位减法

;最高位是

D_{31}

减法之前EAX	AAFF3322H
32位减法	- BB000000H
减法之后EAX	EFF3322H

1110 1111 1111 1111 0011 0011 0010 0010

OF=0, SF=1, ZF=0, PF=1, CF=1

AAFF3322H-BB000000H的最高位发生借位(AA=1001 1001;BB=1010 1010)，实际结果 EFFF3322H 在 $-2^{31} \sim 2^{31}-1$ ，OF=0

小写字母转换为大写可以用SUB

;确认是小写字母, 减20H为大写字母

sub al,20h ;20H='a' - 'A'
;20H=' '(空格字符)

小写=大写+20H
大写=小写-20H

字母	ASCII 值
'A'	41H
'B'	42H
'a'	61H
'b'	62H

本讲总结

- **SUB** 指令是最基本的减法指令
- 支持8、16和32位减法运算
 - ▶ 并按照8、16和32位相应影响状态标志
 - ▶ 但**PF**标志只利用低8位结果

```
SUB dest, src  
; dest ← dest - src
```

汇编语言程序设计

INC、DEC和NEG指令

增量指令INC (increment)

- 只有一个操作数：寄存器或存储单元
- 对操作数加1（增量）再将结果返回原处

i++

INC reg/mem ;加1: $\text{reg/mem} \leftarrow \text{reg/mem} + 1$

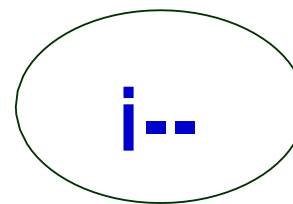
- 用于计数器和地址指针的调整
 - ▶ 不影响进位CF标志，影响其他状态标志位

```
inc ecx  
inc dword ptr [ebx]  
inc wvar
```

```
inc bl  
inc word ptr [esi]  
inc wvar[edi]
```

减量指令DEC (decrement)

- 只有一个操作数：寄存器或存储单元
- 对操作数减1（减量）再将结果返回原处



DEC reg/mem ; 减1: $\text{reg/mem} \leftarrow \text{reg/mem} - 1$

- 用于计数器和地址指针的调整
 - ▶ 不影响进位CF标志，影响其他状态标志位

```
dec cx
```

```
dec byte ptr [ebx]
```

```
dec wvar
```

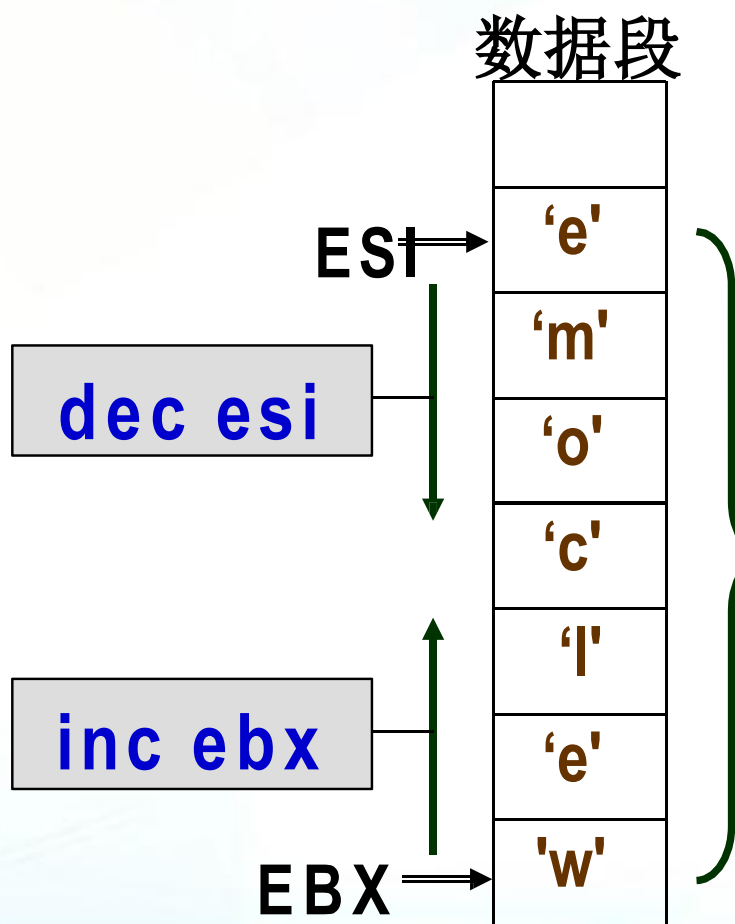
```
dec bl
```

```
dec word ptr [esi]
```

```
dec wvar[edi]
```



INC和DEC用于指针调整和计数增减



```
;数据段  
msg byte 'welcome',0
```

```
mov ecx,(lengthof msg)-1  
;ECX 保存字符个数  
dec ecx ;个数减1  
inc ecx ;个数加1
```

求补指令NEG (negative)

- 对操作数执行求补运算，即用零减去操作数
NEG reg/mem ; $\text{reg/mem} \leftarrow 0 - \text{reg/mem}$
- 对标志的影响与用零作减法的SUB指令一样
- 可用于对负数求补码或由补码求其绝对值

0-i

```
neg al  
neg byte ptr [ebx]  
neg wvar[esi]
```

```
neg ax  
neg word ptr [ebx]  
neg wvar[edi]
```

;已知100的8位编码, 求-100的8位补码

mov al,64h ;AL=64H =100

neg al ;AL=0-64H =9CH = -100

;OF=0, SF=1, ZF=0, PF=1, **CF=1**

;已知-100的32位补码, 求其绝对值 (即100)

mov eax,0fffffff9ch ;EAX=FFFFFFFF9CH = -100

neg eax ;EAX=0-FFFFFFFF9CH =64H =100

;OF=0, SF=0, ZF=0, PF=0, **CF=1**

➤ **INC、DEC和NEG** 是加减运算的辅助指令

- ▶ **INC** 实现指针加1（与**ADD**加1功能相同）、不影响**CF**
- ▶ **DEC** 实现指针减1（与**SUB**减1功能相同）、不影响**CF**
- ▶ **NEG** 进行数据求补（使用**0**减功能实现）

➤ 这些指令都只给出一个操作数位置

- ▶ 既是源操作数，也是目的操作数



算术运算类指令

---加法指令

2. 带进位加法指令ADC

功能：两个操作数相加，再加CF，结果送目的操作数

格式1: `ADC reg,imm/reg/mem` ; $\text{reg} \leftarrow \text{reg} + \text{imm}/\text{reg}/\text{mem} + \text{CF}$

格式2: `ADC mem,imm/reg` ; $\text{mem} \leftarrow \text{mem} + \text{imm}/\text{reg} + \text{CF}$

用于与ADD指令相结合实现多精度数的加法

先将两个操作数的低32位相加（用ADD指令）

再加高位部分、并将进位加到高位（用ADC指令）

例：

<code>mov eax,dword ptr qvar1</code>	;取低32位
<code>add eax,dword ptr qvar2</code>	;加低32位，设置CF
<code>mov edx,dword ptr qvar1+4</code>	;取高32位
<code>adc edx,dword ptr qvar2+4</code>	;加高32位，同时加CF



算术运算类指令

——减法指令

2. 带借位减法指令SBB

功能：目的操作数减去源操作数，**再减CF**，结果送目的操作数

格式1：SBB reg,imm/reg/mem ;reg←reg—imm/reg/mem—**CF**

格式2：SBB mem,imm/reg ;mem←mem—imm/reg—**CF**

用于与SUB指令相结合实现多精度数的减法

先将两个操作数的低32位相减（用SUB指令）

然后减高位部分、并减去借位（用SBB指令）



算术运算类指令

——减法指令

5.比较指令CMP

功能:将目的操作数减去源操作数, **差值不回送目的操作数**, 按照减法结果影响状态标志

CMP reg,imm/reg/mem ;reg—imm/reg/mem

CMP mem,imm/reg ;mem—imm/reg

作用: 1) 根据**标志状态**获知两个操作数的大小关系
2) 给条件转移等指令使用其形成的状态标志

算术运算类指令

——乘法指令和除法指令

IA-32处理器的乘法和除法指令比较特殊

- 针对无符号数和有符号数有各自的指令
有符号数指令前用I (sIgned) 表示
- 隐含使用EAX (和EDX) 寄存器

算术运算类指令

——乘法指令和除法指令

1. 乘法指令

DX.AX = AX × r16/m16
操作数是16位数，AX和其相乘的32位积，高16位存入DX,低16位存入AX
32位也是这么扩展

指令类型	操作数组合及功能	举例
无符号数乘法 MUL src	$AX = AL \times r8/m8$ $DX.AX = AX \times r16/m16$ $EDX.EAX = EAX \times r32/m32$	mul bl imul bx mul dvar
有符号数乘法 IMUL src		
双操作数乘法 IMUL dest,src	$r16 = r16 \times r16/m16/i8/i16$ $r32 = r32 \times r32/m32/i8/i32$	imul eax,10 imul ebx,ecx
三操作数乘法 IMUL dest,src,imm	$r16 = r16/m16 \times i8/i16$ $r32 = r32/m32 \times i8/i32$	imul ax,bx,-2 imul eax,dword ptr [esi+8],5

算术运算类指令

——乘法指令和除法指令

1. 乘法指令

例：计算：A5H × 64H

- 无符号乘法指令：

```
mov al,0a5h
```

```
mov bl,64h
```

```
mul bl
```

$$\text{A5H} (=165) \times 64\text{H} (=100) = 4074\text{H} (=16500)$$

- 有符号乘法指令：

```
mov al,0a5h
```

```
mov bl,64h
```

```
imul bl
```

$$\text{A5H} (= -91) \times 64\text{H} (=100) = \text{DC74H} (= -9100)$$

算术运算类指令

——乘法指令和除法指令

2.除法指令

$AX \leftarrow DX, AX \div r16/m16$ 的商

$DX \leftarrow DX, AX \div r16/m16$ 的余数

除数是16位数，被除数是DX, AX组成的32位数，商和余数分别存储在AX和DX；32位运算也是这么扩展

指令	操作数组合及功能	举例
无符号除法: DIV src	$AL \leftarrow AX \div r8/m8$ 的商 $AH \leftarrow AX \div r8/m8$ 的余数 $AX \leftarrow DX, AX \div r16/m16$ 的商	div bl idiv bx
有符号除法: IDIV src	$DX \leftarrow DX, AX \div r16/m16$ 的余数 $EAX \leftarrow EDX, EAX \div r32/m32$ 的商 $EDX \leftarrow EDX, EAX \div r32/m32$ 的余数	div ebx



算术运算类指令

——乘法指令和除法指令

2. 除法指令

除法指令可能产生除法溢出

- 对DIV指令，除数为0，或者在字节除时商超过8位，在字除时商超过16位，或者双字除时超过32位，则发生除法溢出
- 对IDIV指令，除数为0，或者在字节除时商不在-128~127范围内，在字除时商不在-32768~32767范围内，或者在双字除时商不在 $-2^{31} \sim 2^{31}-1$ 范围内，则发生除法溢出
- 除法错溢出，将产生编号为0的内部中断

算术运算类指令

---零位扩展和符号扩展指令

指令类型	指令	举例
零位扩展	MOVZX r16, r8/m8	movzx di, bvar
	MOVZX r32, r8/m8/r16/m16	movzx eax, ax
符号扩展	MOVSX r16, r8/m8	movsx ax, al
	MOVSX r32, r8/m8/r16/m16	movsx edx, bx

算术运算类指令

——类型转换指令

1. **CBW** ; 字节转换为字

执行操作: **AL → AX; (符号扩展)**

若(AL)的最高有效位为0, 则(AH)= 00H

若(AL)的最高有效位为1, 则(AH)= FFH

2. **CWD** ; 字符扩展为双字

执行操作: **AX → (DX,AX)**

若(AX)的最高有效位为0, 则(DX)= 0000H

若(AX)的最高有效位为1, 则(DX)= FFFFH

3. **CWDE** ; 字符扩展为双字

执行操作: **AX → EAX**

* 无操作数指令

* 隐含对AL 或AX 进行符号扩展

4. **CWQ** ; 双字扩展为四字

执行操作: **EAX → (EDX, EAX)**